Nombre: Luis Rodrigo Martínez Tabernero

Identificador: 1975318

EJERCICIOS MÓDULO 2

0.1)

```
bobovino@Bobovino-laptop:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.5 LTS
Release:        24.04
Codename:       noble
```

Uso Ubuntu – 24.04 instalado en WSL.

```
bobovino@Bobovino-laptop:~$ grep "model name" /proc/cpuinfo | uniq -c
     16 model name      : AMD Ryzen AI 7 350 w/ Radeon 860M
bobovino@Bobovino-laptop:~$
```

Consultando el modelo "AMD Ryzen AI 7 350" en la web de AMD obtenemos:

Processor Architecture	4x Zen 5 , 4x Zen 5c
# of CPU Cores	8
Multithreading (SMT)	Yes
# of Threads	16

-Número de cores: 8

-CPUs lógicas visibles por el SO: 16

-4 núcleos Zen 5 (equivalentes a P-cores)

-4 núcleos Zen 5c (equivalentes a E-cores).

-Esto quiere decir que no todos los núcleos son idénticos. Es una arquitectura híbrida.

-Los núcleos lógicos (16) coinciden con el total de hilos del procesador (8 núcleos x 2 hilos por núcleo mediante SMT). Con WSL tenemos acceso a todo el procesador.

0.2) Tenemos los archivos en la carpeta home de Ubuntu

```
bobovino@Bobovino-laptop:~$ ls
maps.c  mem1.c  step1.c
bobovino@Bobovino-laptop:~$
```

Compilo y se genera con los permisos necesarios:

```
bobovino@Bobovino-laptop:~$ gcc -o step1 step1.c
bobovino@Bobovino-laptop:~$ ls -l step1
-rwxr-xr-x 1 bobovino bobovino 16048 Nov  4 20:58 step1
bobovino@Bobovino-laptop:~$
```

ANÁLISIS DEL CÓDIGO

```
bobovino@Bobovino-laptop:~$ cat step1.c
#include <time.h>
#include <stdlib.h>
#include <error.h>

int
main (int argc, char *argv[])
{
  unsigned int delayus = 0;

  if (argc > 2) error(1, 0, "Invokation error.\nUsage: %s [ delay (in us) ]", argv[0]);

  if (argc == 2)
    delayus = atoi (argv[1]);

  struct timespec delay;
  delay.tv_sec = delayus / 1000000;
  delay.tv_nsec = (delayus % 1000000) * 1000;

  while (1)
    {
      if (delayus)
        nanosleep (&delay, NULL);
    }
}
bobovino@Bobovino-laptop:~$
```

-El programa recibe un parámetro opcional que se convierte en un entero almacenado en "delayus", que representa un retardo en microsegundos.

-Después se convierte a un struct dividido en dos partes: "tv*sec" para la parte entera y "tv*nsec" para el resto en nanosegundos.

-El bucle se comporta tal que si pasamos un valor "delayus>0", invocamos nanosleep y bloqueamos el sistema. Si no se pasa parámetro, el programa entra en un bloqueo y consume toda la CPU hasta que el planificador cambie el turno.

0.3) Como tenemos una arquitectura híbrida y los núcleos P ocupan las primeras posiciones, voy a utilizar el núcleo 0 (en el parámetro -c)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 ./step1 1000
```

1.1)

```
bobovino@Bobovino-laptop:~$ ./step1 100000 & ./step1 10000 & ./step1 1000 & ./step1 100 & ./step1 10 & ./step1 1 &
[1] 657
[2] 658
[3] 659
[4] 660
[5] 661
[6] 662
bobovino@Bobovino-laptop:~$
```

El primer programa ejecutado lo hace en primer plano, y gracias al símbolo ampersand, podemos lanzar otros en segundo plano sin depender de bloqueos de manera concurrente. Aquí vemos el identificador de trabajo y su PID asignado por el kernel.

1.2)

1.2.1) pidstat sirve para monitorizar el rendimiento de Linux. Supervisa el uso de recursos (CPU, memoria, E/S etc) a nivel de proceso/hilo.

-C step1: filtra los procesos cuyo nombre coincida con la cadena "step1"

-l: muestra el nombre del comando en la columna Command para ver su retardo.

-w: estadísticas de cambios de contexto.

5: muestreamos cada 5 segundos

1.2.2) Abro una segunda terminal y ejecuto el comando. Ya estaba instalado.

```
bobovino@Bobovino-laptop:~$ pidstat -C step1 -l -w 5
Linux 6.18.33.2-microsoft-standard-WSL2 (Bobovino-laptop)       11/04/26        _x86_64_        (16 CPU)

22:02:31      UID       PID   cswch/s nvcswch/s  Command
22:02:36     1000       657      9.96      0.00  ./step1 100000
22:02:36     1000       658     98.61      0.00  ./step1 10000
22:02:36     1000       659    936.25      0.00  ./step1 1000
22:02:36     1000       660   5124.70      0.00  ./step1 100
22:02:36     1000       661   9478.69      0.00  ./step1 10
22:02:36     1000       662  10350.60      0.00  ./step1 1
22:02:36     1000       667      0.20      0.00  pidstat -C step1 -l -w 5

22:02:36      UID       PID   cswch/s nvcswch/s  Command
22:02:43     1000       657     10.00      0.00  ./step1 100000
22:02:43     1000       658     99.00      0.00  ./step1 10000
22:02:43     1000       659    935.40      0.00  ./step1 1000
22:02:43     1000       660   5117.00      0.00  ./step1 100
22:02:43     1000       661   9592.00      0.00  ./step1 10
22:02:43     1000       662  10400.60      0.00  ./step1 1
22:02:43     1000       667      0.20      0.00  pidstat -C step1 -l -w 5

22:02:43      UID       PID   cswch/s nvcswch/s  Command
22:02:48     1000       657     10.00      0.00  ./step1 100000
22:02:48     1000       658     99.00      0.00  ./step1 10000
22:02:48     1000       659    937.60      0.00  ./step1 1000
22:02:48     1000       660   5138.80      0.00  ./step1 100
22:02:48     1000       661   9586.20      0.00  ./step1 10
22:02:48     1000       662  10502.40      0.00  ./step1 1
22:02:48     1000       667      0.20      0.00  pidstat -C step1 -l -w 5
```

1.2.3) COLUMNAS

-Tiempo/UID: hora del sistema y el identificador de usuario propietario del proceso

-PID: Identificador del proceso asignado por el kernel

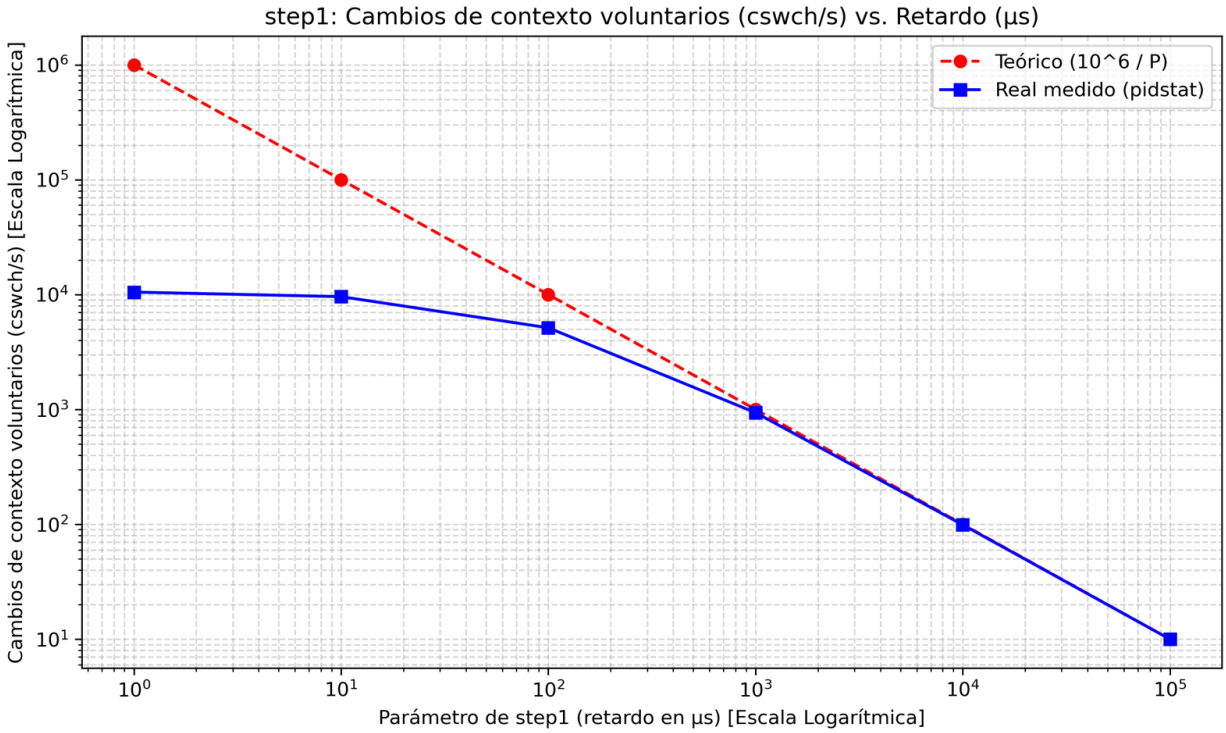
-cswch/s (voluntary context switches per second). Ocurre cuando el proceso cede el procesador antes de agotar su turno.

-nvcswch/s (non-voluntary context switches per second). Ocurre cuando el planificador arrebata la CPU al proceso.

-Command: Nombre del programa ejecutable asociado al proceso

1.3)

Instancia	Parámetro P (microS)	Valor Teórico (10^6/P)	Valor Real medido (cswch/s)	Valor Real medido (nvcswch/s)
./step1 100000	100000	10	10.00	0
./step1 10000	10000	100	99.00	0
./step1 1000	1000	1000	937.60	0
./step1 100	100	10000	5138.80	0
./step1 10	10	100000	9586.20	0
./step1 1	1	1000000	10502.40	0



1.4) Cuando el parámetro es >0, el proceso llama a nanosleep(), pasando el proceso a estar bloqueado. Cuando pasa ese tiempo, el proceso se fasea a Ready. Todo ello de forma voluntaria.

Para valores grandes de P la curva real coincide con la teórica, no así para valores pequeños. Esto es por el overhead del S.O.

En todas las filas nvcswch/s es 0. Por lo tanto, no hay cambios involuntarios.

1.5) Mato los procesos que quedaban con Ctrl + C y el comando killall step1

```
bobovino@Bobovino-laptop:~$ killall step1
[1]   Terminated              ./step1 100000
[2]   Terminated              ./step1 10000
[3]   Terminated              ./step1 1000
[5]-  Terminated              ./step1 10
[4]-  Terminated              ./step1 100
[6]+  Terminated              ./step1 1
```

2.1.1)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 ./step1 &
[1] 1840
bobovino@Bobovino-laptop:~$
```

2.1.2)

```
bobovino@Bobovino-laptop:~$ pidstat -C step1 -l -wu 5
Linux 6.18.33.2-microsoft-standard-WSL2 (Bobovino-laptop)       11/04/26        _x86_64_        (16 CPU)

23:09:06      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
23:09:11     1000      1840  108.17    0.20    0.00    0.40  108.37     0  ./step1
23:09:11     1000      1846    0.00    0.20    0.00    0.00    0.20     8  pidstat -C step1 -l -wu 5

23:09:06      UID       PID   cswch/s nvcswch/s  Command
23:09:11     1000      1840      0.00      4.78  ./step1
23:09:11     1000      1846      0.20      0.00  pidstat -C step1 -l -wu 5
```

Hemos añadido -wu, que añade estadísticas de utilización del procesador por proceso.

2.1.3)

%usr y %CPU: Utilización de la CPU (El bucle infinito se come toda la capacidad)

CPU = 0, es la CPU asignada.

%wait = 0.40%: Prácticamente nulo ya que no hay cola.

Cswch/s = 0: No hay cambios de contexto voluntarios.

Nvcswch/s = 4.78: Cambios involuntarios. Son muy bajos. Serán interrupcioens del reloj del sistema o rutinas del kernel.

2.1.4)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 ./step1 &
[2] 1855
23:22:58      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
23:23:03     1000      1840   50.00    0.00    0.00   50.80   50.00     0  ./step1
23:23:03     1000      1855   50.00    0.20    0.00   50.80   50.20     0  ./step1
23:23:03     1000      1856    0.00    0.20    0.00    0.00    0.20     5  pidstat -C step1 -l -wu 5

23:22:58      UID       PID   cswch/s nvcswch/s  Command
23:23:03     1000      1840      0.00    121.80  ./step1
23:23:03     1000      1855      0.00    121.60  ./step1
23:23:03     1000      1856      0.20      0.00  pidstat -C step1 -l -wu 5
```

CAMBIOS OBSERVADOS

-Se han repartido la CPU a la mitad entre los dos procesos.

- El tiempo de espera ha subido a 50.80% en la cola Ready.

- Aparecen cambios de contexto involuntarios significativos. (sigue sin haber involuntarios)

2.1.5)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 ./step1 & taskset -c 0 ./step1 & taskset -c 0 ./step1 &
[3] 1861
[4] 1862
[5] 1863
23:27:08      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
23:27:13     1000      1840   20.40    0.00    0.00   82.40   20.40     0  ./step1
23:27:13     1000      1855   20.40    0.20    0.00   82.60   20.60     0  ./step1
23:27:13     1000      1856    0.00    0.20    0.00    0.00    0.20     5  pidstat -C step1 -l -wu 5
23:27:13     1000      1861   20.40    0.00    0.00   82.60   20.40     0  ./step1
23:27:13     1000      1862   20.40    0.00    0.00   82.80   20.40     0  ./step1
23:27:13     1000      1863   20.40    0.00    0.00   82.60   20.40     0  ./step1

23:27:08      UID       PID   cswch/s nvcswch/s  Command
23:27:13     1000      1840      0.00     49.40  ./step1
23:27:13     1000      1855      0.00     49.00  ./step1
23:27:13     1000      1856      0.20      0.00  pidstat -C step1 -l -wu 5
23:27:13     1000      1861      0.00     49.00  ./step1
23:27:13     1000      1862      0.00     49.00  ./step1
23:27:13     1000      1863      0.00     49.20  ./step1
```

CAMBIOS OBSERVADOS

- la CPU se reparte al 20% entre todos los procesos. Son 5 procesos así que tiene sentido que 100%/5=20%

-%wait sube a 82.60%. Esto quiere decir que un proceso pasa 4/5 partes de su tiempo esperando en la cola de Ready.

-los cambios involuntarios bajan de 121 a 49. Esto es porque un proceso debe esperar a los otros 4.

2.1.6) wait indica el tiempo que un proceso ha pasado en la cola Ready esperando a que el planificador le asigne CPU.

2.1.7) CÁLCULO CON 2 INSTANCIAS

-Cada proceso sufrió 121.7 cambios involuntarios /s aproximadamente. El total es 121.7 * 2 = 243.4 cambios/s

-q = 1s/243.4 = 1000ms/243.4 = 4.1 ms

CÁLCULO CON 5 INSTANCIAS

-Cambios totales/s = 49.4 + 49 + 49 + 49 + 49.20 = 254.6 cambios/s

-q = 1000 ms /245.6 = 4.07 ms

CONCLUSIÓN

La duración estimada es 4ms, lo que coincide con el valor de compilacion estandar del kernel Linux (CONFIG_HZ = 250). Tick de reloj = 1000/250 = 4ms.

2.1.8)

```
bobovino@Bobovino-laptop:~$ killall step1
[1]   Terminated              taskset -c 0 ./step1
[2]   Terminated              taskset -c 0 ./step1
bobovino@Bobovino-laptop:~$
[3]   Terminated              taskset -c 0 ./step1
[4]-  Terminated              taskset -c 0 ./step1
[5]+  Terminated              taskset -c 0 ./step1
```

2.2.1)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 nice -n 19 ./step1 &
[1] 1938
```

```
00:13:56      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
00:14:02     1000      2133   94.00    0.00    0.00    0.00   94.00     0  ./step1

00:13:56      UID       PID   cswch/s nvcswch/s  Command
00:14:02     1000      1934      0.20      0.00  pidstat -C step1 -l -wu 5
00:14:02     1000      2133      2.00      3.80  ./step1
```

-Usa un 94% de la CPU.

-switch/s despreciable

2.2.2)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 nice -n 19 ./step1 &
[2] 1941
```

```
23:58:46      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
23:58:51     1000      1938   51.40    0.00    0.00   51.40   51.40     0  ./step1
23:58:51     1000      1941   51.40    0.00    0.00   51.40   51.40     0  ./step1

23:58:46      UID       PID   cswch/s nvcswch/s  Command
23:58:51     1000      1934      0.20      0.00  pidstat -C step1 -l -wu 5
23:58:51     1000      1938      0.00    123.60  ./step1
23:58:51     1000      1941      0.00    123.40  ./step1
```

-Se reparten la CPU y el tiempo de respuesta a la mitad, ya que tienen la misma prioridad

-nvcswch/s aumenta a 123.6

2.2.3)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 nice -n 10 ./step1 &
[3] 2028
```

```
00:01:01      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
00:01:06     1000      1934    0.20    0.20    0.00    0.00    0.40     5  pidstat -C step1 -l -wu 5
00:01:06     1000      1938   11.00    0.00    0.00   91.40   11.00     0  ./step1
00:01:06     1000      1941   11.00    0.00    0.00   92.00   11.00     0  ./step1
00:01:06     1000      2028   80.60    0.00    0.00   22.20   80.60     0  ./step1

00:01:01      UID       PID   cswch/s nvcswch/s  Command
00:01:06     1000      1934      0.20      0.00  pidstat -C step1 -l -wu 5
00:01:06     1000      1938      0.00     26.80  ./step1
00:01:06     1000      1941      0.00     26.60  ./step1
00:01:06     1000      2028      0.00     56.20  ./step1
```

Primera vez que un proceso tiene distinta prioridad (10, más alta). Por lo tanto tiene 80.60% del uso del procesador. El resto se lo reparten a la mitad los otros dos.

-El tiempo %wait es menor que el de los otros dos procesos con menor prioridad.

-El switch involuntario es mayor que el de los otros dos procesos.

2.2.4)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 nice -n 0 ./step1 &
[4] 2031
```

```
00:01:49      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
00:01:54     1000      1938    1.40    0.00    0.00  100.60    1.40     0  ./step1
00:01:54     1000      1941    1.40    0.00    0.00  108.20    1.40     0  ./step1
00:01:54     1000      2028   10.20    0.00    0.00   98.60   10.20     0  ./step1
00:01:54     1000      2031   95.40    0.00    0.00   13.20   95.40     0  ./step1

00:01:49      UID       PID   cswch/s nvcswch/s  Command
00:01:54     1000      1934      0.20      0.00  pidstat -C step1 -l -wu 5
00:01:54     1000      1938      0.00      3.00  ./step1
00:01:54     1000      1941      0.00      3.20  ./step1
00:01:54     1000      2028      0.00     23.60  ./step1
00:01:54     1000      2031      0.00     32.40  ./step1
```

Añadimos otro proceso con la prioridad más alta (0) y por lo tanto recibe 95.40% del procesador. El resto se lo queda la mayoría el siguiente proceso con mayor prioridad, y un 1.40% queda para cada uno de los restantes.

-El resto de los procesos pasan la mayor parte del tiempo esperando

-El switch involuntario es más grande en el nuevo proceso.

2.2.5)

```
bobovino@Bobovino-laptop:~$ taskset -c 0 nice -n 0 ./step1 &
[5] 2039
```

```
00:02:20      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
00:02:25     1000      1934    0.00    0.20    0.00    0.00    0.20     5  pidstat -C step1 -l -wu 5
00:02:25     1000      1938    0.60    0.00    0.00   99.80    0.60     0  ./step1
00:02:25     1000      1941    0.80    0.00    0.00  110.20    0.80     0  ./step1
00:02:25     1000      2028    5.40    0.00    0.00  103.40    5.40     0  ./step1
00:02:25     1000      2031   50.60    0.00    0.00   57.40   50.60     0  ./step1
00:02:25     1000      2039   50.60    0.00    0.00   57.60   50.60     0  ./step1

00:02:20      UID       PID   cswch/s nvcswch/s  Command
00:02:25     1000      1934      0.20      0.00  pidstat -C step1 -l -wu 5
00:02:25     1000      1938      0.00      1.60  ./step1
00:02:25     1000      1941      0.00      1.80  ./step1
00:02:25     1000      2028      0.00     12.40  ./step1
00:02:25     1000      2031      0.00    115.00  ./step1
00:02:25     1000      2039      0.00    115.00  ./step1
```

Añadimos un nuevo proceso con la más alta prioridad.

-Procesos con nice 0: Monopilizan la CPU y se la reparten equitativamente.

-Proceso con nice 10: recibe un 5.40% de la CPU.

-Procesos con nice 19: Reciben una fracción marginal de tiempo (sin llegar a la inanición)

CÁLCULO DE QUANTUM CON NICE 0:

-115 cambios por segundo y 0.506 s de CPU/segundo real

Q = 0.506/115 = 0.0044 s = 4.4ms

CÁLCULO DE QUANTUM CON NICE 10:

-12.40 cambios/s y 0.054 s de CPU

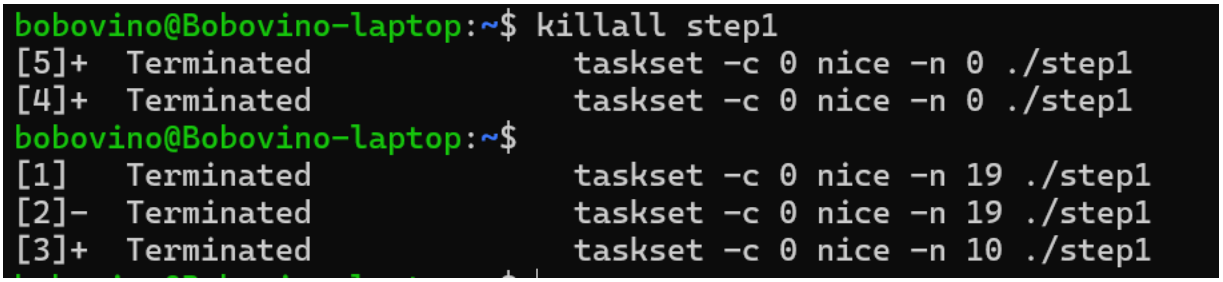
Q= 0.054/12.40= 0.0043 = 4.3ms

CÁLCULO DE QUANTUM CON NICE 19:

Q = 0.007s / 1.70 turnos = 4.1 ms

Todos rondan el estándar de 4ms

2.3)



EJERCICIOS MÓDULO 3

1)

```
bobovino@Bobovino-laptop:~$ cat /proc/self/maps
61d14e00b000-61d14e00d000 r--p 00000000 08:30 1461                       /usr/bin/cat
61d14e00d000-61d14e012000 r-xp 00002000 08:30 1461                       /usr/bin/cat
61d14e012000-61d14e014000 r--p 00007000 08:30 1461                       /usr/bin/cat
61d14e014000-61d14e015000 r--p 00008000 08:30 1461                       /usr/bin/cat
61d14e015000-61d14e016000 rw-p 00009000 08:30 1461                       /usr/bin/cat
61d15a5cc000-61d15a5ed000 rw-p 00000000 00:00 0                          [heap]
7d1a15800000-7d1a15828000 r--p 00000000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d1a15828000-7d1a159b1000 r-xp 00028000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d1a159b1000-7d1a15a00000 r--p 001b1000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d1a15a00000-7d1a15a04000 r--p 001ff000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d1a15a04000-7d1a15a06000 rw-p 00203000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d1a15a06000-7d1a15a13000 rw-p 00000000 00:00 0
7d1a15a21000-7d1a15a43000 rw-p 00000000 00:00 0
7d1a15a43000-7d1a15a9c000 r--p 00000000 08:30 12253                      /usr/lib/locale/C.utf8/LC_CTYPE
7d1a15a9c000-7d1a15a9d000 r--p 00000000 08:30 12259                      /usr/lib/locale/C.utf8/LC_NUMERIC
7d1a15a9d000-7d1a15a9e000 r--p 00000000 08:30 12262                      /usr/lib/locale/C.utf8/LC_TIME
7d1a15a9e000-7d1a15a9f000 r--p 00000000 08:30 12252                      /usr/lib/locale/C.utf8/LC_COLLATE
7d1a15a9f000-7d1a15aa0000 r--p 00000000 08:30 12257                      /usr/lib/locale/C.utf8/LC_MONETARY
7d1a15aa0000-7d1a15aa1000 r--p 00000000 08:30 12256                      /usr/lib/locale/C.utf8/LC_MESSAGES/S
YS_LC_MESSAGES
7d1a15aa1000-7d1a15aa8000 r--s 00000000 08:30 52104                      /usr/lib/x86_64-linux-gnu/gconv/gcon
v-modules.cache
7d1a15aa8000-7d1a15aab000 rw-p 00000000 00:00 0
7d1a15aab000-7d1a15aac000 r--p 00000000 08:30 12260                      /usr/lib/locale/C.utf8/LC_PAPER
7d1a15aac000-7d1a15aad000 r--p 00000000 08:30 12258                      /usr/lib/locale/C.utf8/LC_NAME
7d1a15aad000-7d1a15aae000 r--p 00000000 08:30 12251                      /usr/lib/locale/C.utf8/LC_ADDRESS
7d1a15aae000-7d1a15aaf000 r--p 00000000 08:30 12261                      /usr/lib/locale/C.utf8/LC_TELEPHONE
7d1a15aaf000-7d1a15ab0000 r--p 00000000 08:30 12255                      /usr/lib/locale/C.utf8/LC_MEASUREMEN
T
7d1a15ab0000-7d1a15ab1000 r--p 00000000 08:30 12254                      /usr/lib/locale/C.utf8/LC_IDENTIFICA
TION
7d1a15ab1000-7d1a15ab3000 rw-p 00000000 00:00 0
7d1a15ab3000-7d1a15ab7000 r--p 00000000 00:00 0                          [vvar]
7d1a15ab7000-7d1a15ab9000 r--p 00000000 00:00 0                          [vvar_vclock]
7d1a15ab9000-7d1a15abb000 r-xp 00000000 00:00 0                          [vdso]
7d1a15abb000-7d1a15abc000 r--p 00000000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
7d1a15abc000-7d1a15ae7000 r-xp 00001000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
7d1a15ae7000-7d1a15af1000 r--p 0002c000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
7d1a15af1000-7d1a15af3000 r--p 00036000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
7d1a15af3000-7d1a15af5000 rw-p 00038000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
7ffc9a984000-7ffc9a9a5000 rw-p 00000000 00:00 0                          [stack]
bobovino@Bobovino-laptop:~$
```

1.1) El archivo expone el el mapa del espacio lógico virtual de direcciones del proceso que lo lee (cat). Cada línea corresponde con una región contigua de memoria virtual.

-address: dirección_*inicial-dirección_final en hx*.

-Permisos: cuatro caracteres, lectura (r/-), escritura (w/-), ejecución (x/-) y modo privado (p/s) o compartido.

-Desplazamiento (offset): distancia en bytes desde el principio del archivo físico mapeado.

-Dispositivo: (major:minor) donde reside el binario.

-Inodo: Identificador del archivo dentro del sistema de ficheros (0 significa anónimo)

-Ruta: mapeado en disco o pseudónimos del kernel entre corchetes.

1.2) /usr/bin/cat

r—p: (00000000 y 00080000) Solo lectura. No puede modificarse.

r-xp:(00020000): Segmento de lectura y ejecución. De código(.text) lenguaje máquina.

Rw-p(00090000): Segmento de datos (.dataa y .bss). Permite lectoescritura para almacenar variables que cambian en tiempo de ejecución.

1.3) No se puede deducir la ubicación de la pila del proceso en memoria física. En la línea [stack] el rango visible corresponde a las direcciones virtuales/lógicas. Estas direcciones están traducidas por el S.O. y la MMU. Están paginadas de una manera no contigua.

1.4)Ejecuto de nuevo:

```
bobovino@Bobovino-laptop:~$ cat /proc/self/maps
6126e7e73000-6126e7e75000 r--p 00000000 08:30 1461                       /usr/bin/cat
6126e7e75000-6126e7e7a000 r-xp 00002000 08:30 1461                       /usr/bin/cat
6126e7e7a000-6126e7e7c000 r--p 00007000 08:30 1461                       /usr/bin/cat
6126e7e7c000-6126e7e7d000 r--p 00008000 08:30 1461                       /usr/bin/cat
6126e7e7d000-6126e7e7e000 rw-p 00009000 08:30 1461                       /usr/bin/cat
6126ffda3000-6126ffdc4000 rw-p 00000000 00:00 0                          [heap]
7d59eca00000-7d59eca28000 r--p 00000000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d59eca28000-7d59ecbb1000 r-xp 00028000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d59ecbb1000-7d59ecc00000 r--p 001b1000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d59ecc00000-7d59ecc04000 r--p 001ff000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d59ecc04000-7d59ecc06000 rw-p 00203000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
7d59ecc06000-7d59ecc13000 rw-p 00000000 00:00 0
7d59ecc71000-7d59ecc93000 rw-p 00000000 00:00 0
7d59ecc93000-7d59eccec000 r--p 00000000 08:30 12253                      /usr/lib/locale/C.utf8/LC_CTYPE
7d59eccec000-7d59eced000 r--p 00000000 08:30 12259                       /usr/lib/locale/C.utf8/LC_NUMERIC
7d59eced000-7d59eccee000 r--p 00000000 08:30 12262                       /usr/lib/locale/C.utf8/LC_TIME
7d59eccee000-7d59eccef000 r--p 00000000 08:30 12252                      /usr/lib/locale/C.utf8/LC_COLLATE
7d59eccef000-7d59eccf0000 r--p 00000000 08:30 12257                      /usr/lib/locale/C.utf8/LC_MONETARY
7d59eccf0000-7d59eccf1000 r--p 00000000 08:30 12256                      /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_ME
SSAGES
7d59eccf1000-7d59eccf8000 r--s 00000000 08:30 52104                      /usr/lib/x86_64-linux-gnu/gconv/gconv-module
s.cache
7d59eccf8000-7d59eccfb000 rw-p 00000000 00:00 0
7d59eccfb000-7d59eccfc000 r--p 00000000 08:30 12260                      /usr/lib/locale/C.utf8/LC_PAPER
7d59eccfc000-7d59eccfd000 r--p 00000000 08:30 12258                      /usr/lib/locale/C.utf8/LC_NAME
7d59eccfd000-7d59eccfe000 r--p 00000000 08:30 12251                      /usr/lib/locale/C.utf8/LC_ADDRESS
7d59eccfe000-7d59eccff000 r--p 00000000 08:30 12261                      /usr/lib/locale/C.utf8/LC_TELEPHONE
7d59eccff000-7d59ecd00000 r--p 00000000 08:30 12255                      /usr/lib/locale/C.utf8/LC_MEASUREMENT
7d59ecd00000-7d59ecd01000 r--p 00000000 08:30 12254                      /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7d59ecd01000-7d59ecd03000 rw-p 00000000 00:00 0
7d59ecd03000-7d59ecd07000 r--p 00000000 00:00 0                          [vvar]
7d59ecd07000-7d59ecd09000 r--p 00000000 00:00 0                          [vvar_vclock]
7d59ecd09000-7d59ecd0b000 r-xp 00000000 00:00 0                          [vdso]
7d59ecd0b000-7d59ecd0c000 r--p 00000000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7d59ecd0c000-7d59ecd37000 r-xp 00001000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7d59ecd37000-7d59ecd41000 r--p 0002c000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7d59ecd41000-7d59ecd43000 r--p 00036000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7d59ecd43000-7d59ecd45000 rw-p 00038000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7ffc7c8cd000-7ffc7c8ee000 rw-p 00000000 00:00 0                          [stack]
bobovino@Bobovino-laptop:~$
```

Las direcciones cambian porque se activa el ASLR (Address Space Layout Randomization).

Es una técnica de seguridad que dificulta la explotación de vulnerabilidades de memoria para que un hacker no pueda predecir dónde se ubicarían las variables críticas.

2.1)

```
bobovino@Bobovino-laptop:~$ gcc -o maps maps.c
./maps
pid=2332
stack=0x7ffce71627b4
malloc=0x5562ae19f6b0
main=0x55629e2621b9
printf=0x765adf660100
```

2.2) Voy a una segunda terminal y ejecuto:

```
bobovino@Bobovino-laptop:~$ cat /proc/2332/maps
55629e261000-55629e262000 r--p 00000000 08:30 14888                      /home/bobovino/maps
55629e262000-55629e263000 r-xp 00001000 08:30 14888                      /home/bobovino/maps
55629e263000-55629e264000 r--p 00002000 08:30 14888                      /home/bobovino/maps
55629e264000-55629e265000 r--p 00002000 08:30 14888                      /home/bobovino/maps
55629e265000-55629e266000 rw-p 00003000 08:30 14888                      /home/bobovino/maps
5562ae19f000-5562ae1c0000 rw-p 00000000 00:00 0                          [heap]
765adf600000-765adf628000 r--p 00000000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
765adf628000-765adf7b1000 r-xp 00028000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
765adf7b1000-765adf800000 r--p 001b1000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
765adf800000-765adf804000 r--p 001ff000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
765adf804000-765adf806000 rw-p 00203000 08:30 52115                      /usr/lib/x86_64-linux-gnu/libc.so.6
765adf806000-765adf813000 rw-p 00000000 00:00 0
765adf93f000-765adf942000 rw-p 00000000 00:00 0
765adf948000-765adf94a000 rw-p 00000000 00:00 0
765adf94a000-765adf94e000 r--p 00000000 00:00 0                          [vvar]
765adf94e000-765adf950000 r--p 00000000 00:00 0                          [vvar_vclock]
765adf950000-765adf952000 r-xp 00000000 00:00 0                          [vdso]
765adf952000-765adf953000 r--p 00000000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
765adf953000-765adf97e000 r-xp 00001000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
765adf97e000-765adf988000 r--p 0002c000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
765adf988000-765adf98a000 r--p 00036000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
765adf98a000-765adf98c000 rw-p 00038000 08:30 52112                      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so
.2
7ffce7143000-7ffce7164000 rw-p 00000000 00:00 0                          [stack]
bobovino@Bobovino-laptop:~$
```

COMPARACIÓN POR FILAS

-stack = 0x7ffce71627b4. -> Cae en el rango de [stack] en la segunda terminal. Es la variable local int foo. Todas las variables locales están en el segmento de activación de la pila.

-malloc = 0x5562ae19f6b0 ->Cae en el rango de [heap] en la segunda terminal. Es memoria dinámica reservada en tiempo de ejecución.

-main = 0x 55629e2621b9 -> Cae en el rango con permisos r-xp sobre el ejecutable /home/bobovino/maps. Apunta a la función main dentro de la sección de código (.text)

-printf = 0x765adf660100 -> Cae en el rango con permisos r-xp sobre la biblioteca /usr/lib/x86_64-linux-gnu/libc.so.6

Rutina del sistema que pertenece a una biblioteca de C cargada dinámicamente.

2.3) El rango reservado para [heap] es: 0x5562ae19f000 --> 0x5562ae1c0000.

Esto es mucho más que 1 solo byte, que es lo que pide malloc.

Motivos: No se asigna memoria byte a byte, sino en múltiplos de páginas de memoria virtual (normalmente de 4kb en 4kb).

Además, el asignador de glic/ptmalloc funciona así: cada llamada al sistema (brk o mmap) para pedir memoria al kernel cambiar el contexto a modo privilegiado. Inicialmente, el gestor de memoria de C pide un bloque grande de memoria y después va atendiendo las reservas pequeñas.

3)Funciona bien

```
bobovino@Bobovino-laptop:~$ gcc -o mem1 mem1.c
bobovino@Bobovino-laptop:~$ ./mem1
Passwd? hola
Passwd? asdf
Passwd? uoc
Passwd OK
bobovino@Bobovino-laptop:~$
```

Y en la segunda ejecución hay overflow

```
bobovino@Bobovino-laptop:~$ ./mem1
Passwd? AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Passwd? uoc
Passwd? uoc
```

3.1) El código es el siguiente:

```
bobovino@Bobovino-laptop:~$ cat mem1.c
#include <stdio.h>
#include <string.h>

struct
{
  char input[8];
  char test[8];
} data;

int
main (int argc, char *argv[])
{

  sprintf (data.test, "uoc");

  do
    {
      printf ("Passwd? ");
      scanf ("%s", data.input);
    }
  while (strcmp (data.input, data.test) != 0);

  printf ("Passwd OK\n");
  return (0);
}
bobovino@Bobovino-laptop:~$
```

Al arrancar, el programa guarda uoc dentro de data.test

Luego, pide la contraseña y la guarda sin comprobar cuánto espacio tiene reservado el vector, por lo que hay posibilidad de overflow.

Si introducimos una cadena muy larga de letras, los primeros 8 caracteres llenan data.input pero los siguientes invaden data.test. Porl o tanto, la contraseña se convierte en "abc" o en mi captura en muchas A.

3.2) Hay que limitar la cantidad de bytes que se pueden escribir en el búfer data.input. Podemos limitar el formato a 7 caracteres (+ el carácter nulo)

```
scanf("%7s", data.input);
```

Habra mejores prácticas, pero bueno, así sirve para nuestro ejemplo.

EJERCICIOS MÓDULO 4

```
bobovino@Bobovino-laptop:~$ tty
/dev/pts/0
```

1)

El primer comando imprime la lista directamente en pantalla:

```
bobovino@Bobovino-laptop:~$ ls -l /dev
total 0
crw-r--r-- 1 root root     10, 235 Nov  4 20:48 autofs
drwxr-xr-x 2 root root         620 Nov  4 20:48 block
drwxr-xr-x 2 root root         140 Nov  4 20:48 bsg
crw------- 1 root root     10, 234 Nov  4 20:48 btrfs-control
drwxr-xr-x 2 root root        2900 Nov  4 20:48 char
crw------- 1 root root      5,   1 Nov  5 00:35 console
lrwxrwxrwx 1 root root          11 Nov  4 20:48 core -> /proc/kcore
crw------- 1 root root     10, 260 Nov  4 20:48 cpu_dma_latency
crw------- 1 root root     10, 203 Nov  4 20:48 cuse
drwxr-xr-x 6 root root         120 Nov  4 20:48 disk
crw-rw-rw- 1 root root     10, 258 Nov  4 20:48 dxg
lrwxrwxrwx 1 root root          13 Nov  4 20:48 fd -> /proc/self/fd
crw-rw-rw- 1 root root      1,   7 Nov  4 20:48 full
crw-rw-rw- 1 root root     10, 229 Nov  4 20:48 fuse
crw------- 1 root root     10, 228 Nov  4 20:48 hpet
drwxr-xr-x 2 root root           0 Nov  4 20:48 hugepages
crw------- 1 root root    229,   0 Nov  4 20:48 hvc0
crw------- 1 root root    229,   1 Nov  5 00:35 hvc1
crw--w---- 1 root tty     229,   2 Nov  4 20:48 hvc2
crw------- 1 root root    229,   3 Nov  4 20:48 hvc3
crw------- 1 root root    229,   4 Nov  4 20:48 hvc4
crw------- 1 root root    229,   5 Nov  4 20:48 hvc5
crw------- 1 root root    229,   6 Nov  4 20:48 hvc6
crw------- 1 root root    229,   7 Nov  4 20:48 hvc7
crw------- 1 root root     10, 183 Nov  4 20:48 hwrng
lrwxrwxrwx 1 root root          12 Nov  4 20:48 initctl -> /run/initctl
crw-r--r-- 1 root root      1,  11 Nov  4 20:48 kmsg
crw-rw---- 1 root kvm      10, 232 Nov  4 20:48 kvm
lrwxrwxrwx 1 root root          28 Nov  4 20:48 log -> /run/systemd/journal/dev-log
crw-rw---- 1 root disk     10, 237 Nov  4 20:48 loop-control
brw-rw---- 1 root disk      7,   0 Nov  4 20:48 loop0
brw-rw---- 1 root disk      7,   1 Nov  4 20:48 loop1
brw-rw---- 1 root disk      7,   2 Nov  4 20:48 loop2
brw-rw---- 1 root disk      7,   3 Nov  4 20:48 loop3
brw-rw---- 1 root disk      7,   4 Nov  4 20:48 loop4
brw-rw---- 1 root disk      7,   5 Nov  4 20:48 loop5
brw-rw---- 1 root disk      7,   6 Nov  4 20:48 loop6
brw-rw---- 1 root disk      7,   7 Nov  4 20:48 loop7
drwxr-xr-x 2 root root          60 Nov  4 20:48 mapper
crw------- 1 root root     10, 227 Nov  4 20:48 mcelog
crw-r----- 1 root kmem      1,   1 Nov  4 20:48 mem
crw------- 1 root root     10, 220 Nov  4 20:48 mptctl
drwxrwxrwt 2 root root          40 Nov  4 20:48 mqueue
drwxr-xr-x 2 root root          60 Nov  4 20:48 net
crw-rw-rw- 1 root root      1,   3 Nov  4 20:48 null
crw------- 1 root root     10, 144 Nov  4 20:48 nvram
crw------- 1 root root    108,   0 Nov  4 20:48 ppp
crw-rw-rw- 1 root tty       5,   2 Nov  5 02:16 ptmx
crw------- 1 root root    246,   0 Nov  4 20:48 ptp0
lrwxrwxrwx 1 root root           4 Nov  4 20:48 ptp_hyperv -> ptp0
```

Los otros dos comandos no imprimen nada, sino que imprimen en el archivo output y el archivo que representa la otra sesión abierta

```
bobovino@Bobovino-laptop:~$ ls -l /dev > output
bobovino@Bobovino-laptop:~$ ls -l /dev > /dev/pts/0
```

Archivo output:

```
bobovino@Bobovino-laptop:~$ cat output
total 0
crw-r--r-- 1 root root     10, 235 Nov  4 20:48 autofs
drwxr-xr-x 2 root root          620 Nov  4 20:48 block
drwxr-xr-x 2 root root          140 Nov  4 20:48 bsg
crw------- 1 root root     10, 234 Nov  4 20:48 btrfs-control
drwxr-xr-x 2 root root         2900 Nov  4 20:48 char
crw------- 1 root root      5,   1 Nov  5 00:35 console
lrwxrwxrwx 1 root root           11 Nov  4 20:48 core -> /proc/kcore
crw------- 1 root root     10, 260 Nov  4 20:48 cpu_dma_latency
crw------- 1 root root     10, 203 Nov  4 20:48 cuse
drwxr-xr-x 6 root root          120 Nov  4 20:48 disk
crw-rw-rw- 1 root root     10, 258 Nov  4 20:48 dxg
lrwxrwxrwx 1 root root           13 Nov  4 20:48 fd -> /proc/self/fd
crw-rw-rw- 1 root root      1,   7 Nov  4 20:48 full
crw-rw-rw- 1 root root     10, 229 Nov  4 20:48 fuse
```

La otra sesión:

```
bobovino@Bobovino-laptop:~$ total 0
crw-r--r-- 1 root root     10, 235 Nov  4 20:48 autofs
drwxr-xr-x 2 root root          620 Nov  4 20:48 block
drwxr-xr-x 2 root root          140 Nov  4 20:48 bsg
crw------- 1 root root     10, 234 Nov  4 20:48 btrfs-control
drwxr-xr-x 2 root root         2900 Nov  4 20:48 char
crw------- 1 root root      5,   1 Nov  5 00:35 console
lrwxrwxrwx 1 root root           11 Nov  4 20:48 core -> /proc/kcore
crw------- 1 root root     10, 260 Nov  4 20:48 cpu_dma_latency
crw------- 1 root root     10, 203 Nov  4 20:48 cuse
drwxr-xr-x 6 root root          120 Nov  4 20:48 disk
crw-rw-rw- 1 root root     10, 258 Nov  4 20:48 dxg
```

2) No hay diferencia. En UNIX todo está abstraido y "todo es un fichero" y los dispositivos son independientes.

3) Normalmente, en esa posición se muestra el tamaño del fichero en bytes. Como todo se representa como un fichero y a veces no tiene sentido hablar de almacenamiento real, tenemos c y b.

En el caso de c (character device) o b (block device) el S.O. muestra:

-Major number: Identifica el driver en el kernel.

-Minor number (tras la coma): Instancia o partición que maneja el driver.